

Default Interface Methods in C# with Examples

In this article, I am going to discuss **Default Interface Methods in C#** with Examples. Please read our previous article, where we discussed C# 8 new Feature **Readonly Structs in C#** with Examples. Earlier, one major difference between the Abstract Class and Interface was that we could not add a default method in the interface which is possible with the abstract class. Now with C# 8.0, we can add the default method to the interface.

Default Interface Methods in C#

Before C# 8.0 interfaces only contain the declaration of the members (Methods, Properties, Events, and Indexers), but from C# 8.0, it is possible to add members as well as their implementation within the interface.

Now we are allowed to add a method with its implementation within the interface without breaking the existing implementation of the interface, such type of method is known as the default interface method (also known as the virtual extension method).

The main benefit of the Default Interface Method is that it allows us to add new functionality to the interfaces of our libraries and ensure backward compatibility with code written for older versions of those interfaces.

Allowed in the Interface in C#:

1. A body for a method or indexer, property, or an event accessor
2. Private, protected, internal, public, virtual, abstract, sealed, static, extern
3. Static fields
4. Static methods, properties, indexers, and events.
5. Explicit access modifiers with default access are public

Not allowed in the interface in C#:

1. Instance state, instance fields, instance auto-properties
2. override keyword is currently not possible, but this might be changed in C# 9

Note: Please use this feature carefully. Otherwise, it can easily lead to violating the single responsibility principles.

Syntax to Provide Default Implementation for Interface Method in C#:

```
interface IDefaultInterfaceMethod
{
    public void DefaultMethod()
    {
        Console.WriteLine("I am a default method in the interface!");
    }
}
```

Example to Understand Default Interface Methods in C#

For a better understanding of the Default Implementation Interface Method in C#, please have a look at the following example. If you look at the below code, then you will see that the interface has a method i.e. DefaultMethod with implementation, and the implementer class is AnyClass which has no idea about this default implementation interface method. The following code is self-explained, so please go through the comment lines for a better understanding.

```
using System;
namespace DefaultInterfaceMethodsDemo
{
    //Interface with Default Method
    interface IDefaultInterfaceMethod
    {
        //Default Implementation Method
        public void DefaultMethod()
        {
            Console.WriteLine("I am a default method in the interface!");
        }
    }
    //Class Inheriting from an Interface which have one Default Implementation
    class AnyClass : IDefaultInterfaceMethod
    {
        //AnyClass Have No Idea about the Default Implementation Interface Method
    }
    class Program
    {
        static void Main(string[] args)
        {
            //Create an Instance of Child class and store it on the Parent Class
            Reference Variable
            IDefaultInterfaceMethod anyClass = new AnyClass();
            //Call the Default Implementation Interface Method
            anyClass.DefaultMethod();
            Console.ReadKey();
        }
    }
}
```

Output:

```
I am a default method in the interface!
```

As we discussed, AnyClass has no idea about the Default Implementation Interface Method. To prove this, Change the IDefaultInterfaceMethod to AnyClass while creating the instance of AnyClass inside the Main method as shown in the below code.

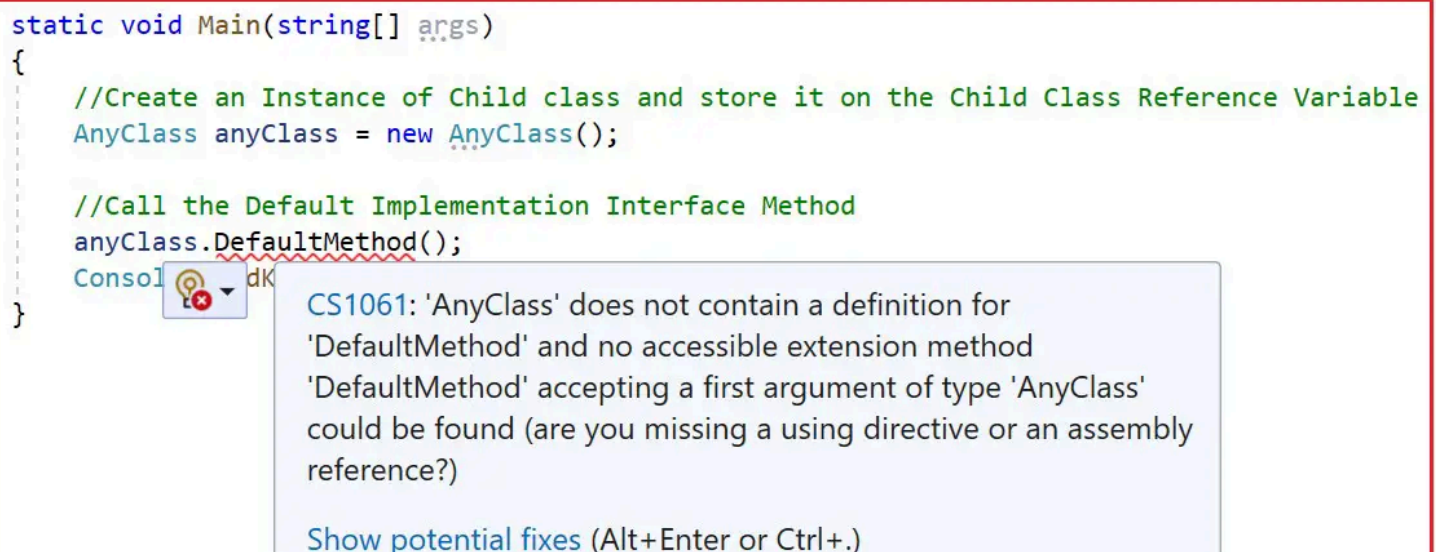
```
using System;
namespace DefaultInterfaceMethodsDemo
```

```

{
    //Interface with Deafult Method
    interface IDefaultInterfaceMethod
    {
        //Default Implementation Method
        public void DefaultMethod()
        {
            Console.WriteLine("I am a default method in the interface!");
        }
    }
    //Class Inheriting from an Interface which have one Default Implementation
    class AnyClass : IDefaultInterfaceMethod
    {
        //AnyClass Have No Idea about the Default Implementation Interface Met
    }
    class Program
    {
        static void Main(string[] args)
        {
            //Create an Instance of Child class and store it on the Child Class
            Variable AnyClass anyClass = new AnyClass();
            //Call the Default Implementation Interface Method
            anyClass.DefaultMethod();
            Console.ReadKey();
        }
    }
}

```

Once you do the above changes, you will get a compile-time error **CS1061: 'AnyClass' does not contain a definition for 'DefaultMethod', and no accessible extension method 'DefaultMethod' accepting a first argument of type 'AnyClass' could be found (are you missing a using directive or an assembly reference?)** as shown in the below image.



```

static void Main(string[] args)
{
    //Create an Instance of Child class and store it on the Child Class Reference Variable
    AnyClass anyClass = new AnyClass();

    //Call the Default Implementation Interface Method
    anyClass.DefaultMethod();
    Console.ReadKey();
}

```

CS1061: 'AnyClass' does not contain a definition for 'DefaultMethod' and no accessible extension method 'DefaultMethod' accepting a first argument of type 'AnyClass' could be found (are you missing a using directive or an assembly reference?)

Show potential fixes (Alt+Enter or Ctrl+.)

This error message proves that the inherited class does not know anything about the default implementation method of an interface in C#. And this is the difference between Abstract Class Implementation Methods and Concrete Class Implementation Methods. So, here, if we use an abstract class instead of an interface, then using the AnyClass object, we can access the DefaultMethod. To prove this modify the example code as follows to use an abstract class instead of an interface.

```
using System;
namespace DefaultInterfaceMethodsDemo
{
    //Abstract Class with Deaful Method
    abstract class DefaultAbstractClassMethod
    {
        //Default Implementation Method
        public void DefaultMethod()
        {
            Console.WriteLine("I am a default method in the Abstract Class!");
        }
    }
    //Class Inheriting from an Abstract class
    class AnyClass : DefaultAbstractClassMethod
    {
        //AnyClass Have Idea about the Default Implementation Abstract Class Method
    }
    class Program
    {
        static void Main(string[] args)
        {
            //Create an Instance of Child class and store it on the Child Class Variable
            AnyClass anyClass = new AnyClass();
            //Call the Default Implementation Abstract Class Method
            anyClass.DefaultMethod();
            Console.ReadKey();
        }
    }
}
```

Output: I am a default method in the Abstract Class!

Modifiers with Default Interface Methods in C#

Now, an interface with C# 8 is extended to accept modifiers such as protected, internal, public, and virtual. By default, the default methods of an interface are virtual. If you want then you can also make them sealed and private by using the sealed or private modifier.

Similarly, if you are not providing implementation to interface methods, then by default they are going to be abstract and public. For a better understanding of Modifiers with Default Interface Methods in C#, please have a look at the below example. The following example code is self-explained, so please go through the comment lines for a better understanding.

```

using System;
namespace DefaultInterfaceMethodsDemo
{
    interface IDefaultInterfaceMethod
    {
        // By default, this method is virtual. The virtual keyword is not required here
        virtual void DefaultMethod()
        {
            Console.WriteLine("I am a default method in the interface!");
        }
        // By default, this method is abstract and public, so the abstract and public
        keyword not required here
        public abstract void Sum();
    }
    interface IOVERRIDEDefaultInterfaceMethod : IDefaultInterfaceMethod
    {
        //Overriding DefaultMethod in Child Interface
        void IDefaultInterfaceMethod.DefaultMethod()
        {
            Console.WriteLine("I am an overridden default method!");
        }
    }
    class AnyClass : IDefaultInterfaceMethod, IOVERRIDEDefaultInterfaceMethod
    {
        //Providing Implementation for Interface Abstract Method
        public void Sum()
        {
        }
    }
    class Program
    {
        static void Main(string[] args)
        {
            //Create an Instance of AnyClass and store it inside the
            IDefaultInterfaceMethod Parent Reference Variable
            IDefaultInterfaceMethod anyClass = new AnyClass();
            //Call to DefaultMethod will Execute from the IOVERRIDEDefaultInterface
            Interface
            anyClass.DefaultMethod();
            //Create an Instance of AnyClass and store it inside the
            IOVERRIDEDefaultInterfaceMethod Parent Reference Variable
            IOVERRIDEDefaultInterfaceMethod anyClassOverridden = new AnyClass();
            //Call to DefaultMethod will Execute from the IOVERRIDEDefaultInterface
            Interface
            anyClassOverridden.DefaultMethod();
            Console.ReadKey();
        }
    }
}

```

Inside the `IDefaultInterfaceMethod` interface, we defined one default implementation method i.e. `DefaultMethod` using the `Virtual` keyword. Using the `virtual` keyword is optional there as by default all the default implementation methods of an interface are by virtual. Then we declared the `Sum` method with the `public` and `abstract` keywords and here the `public` and `abstract` keywords are optional as by default all the interface methods are `public` and `abstract` if we are not providing the implementation.

Then in the `IOverrideDefaultInterfaceMethod` interface, we override the `DefaultMethod` method. As the `DefaultMethod` method is a virtual method in the `IDefaultInterfaceMethod` interface, we can override it. Then in the child class i.e. `AnyClass`, we implement both the interfaces and provide an implementation of the abstract `Sum` method.

Now, run the above code and you will get the following output. In both the call to the `DefaultMethod` is executed from the `IOverrideDefaultInterfaceMethod` interface. This is because, at the time of Program execution, first, it will check the `DefaultMethod` body within the `AnyClass`, and if found it will execute from `AnyClass`. If not found, then it will check its immediate parent interface which is `IOverrideDefaultInterfaceMethod` and from this class, the method is going to be executed.

```
I am an overridden default method!  
I am an overridden default method!
```

Access Modifier Now Allowed in Overridden Method of an Interface in C#

The explicit access modifiers in the overridden method of an Interface are not allowed in C#. Let us understand this with an example. Please have a look at the below example code. In the below example, inside the **`IDefaultInterfaceMethod`** interface, we have provided a default implementation to the **`DefaultMethod`**. And in the **`IOverrideDefaultInterfaceMethod`** interface, we are overriding the **`DefaultMethod`** and trying to provide the access modifier as `public` which is not allowed.

```
using System;  
namespace DefaultInterfaceMethodsDemo  
{  
    interface IDefaultInterfaceMethod  
    {  
        // By default, this method is virtual. The virtual keyword is not required  
        virtual void DefaultMethod()  
        {  
            Console.WriteLine("I am a default method in the interface!");  
        }  
        // By default, this method is abstract, so the abstract keyword not required  
        abstract void Sum();  
    }  
    interface IOverrideDefaultInterfaceMethod : IDefaultInterfaceMethod  
    {  
        //Overriding DefaultMethod in Child Interface  
        //Access Specifier is not allowed  
        public void IDefaultInterfaceMethod.DefaultMethod()  
        {  
            Console.WriteLine("I am an overridden default method!");  
        }  
    }  
}  
class Program
```

```

{
    static void Main(string[] args)
    {
        Console.ReadKey();
    }
}

```

Once you do the above changes, you will get a compile-time error **The modifier public is not valid for this item** as shown in the below image.

```

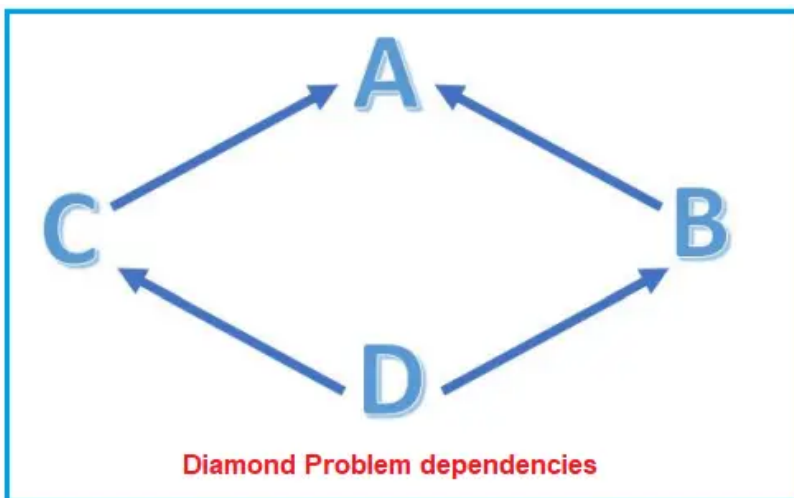
interface IOverrideDefaultInterfaceMethod : IDefaultInterfaceMethod
{
    //Overriding DefaultMethod in Child Interface
    3 references
    public void IDefaultInterfaceMethod.DefaultMethod()
    {
        Console.WriteLine("I am an overri
    }
}

```

void IOverrideDefaultInterfaceMethod.DefaultMethod()
CS0106: The modifier 'public' is not valid for this item

Diamond Problem with Multiple Inheritance in C#

We can get the diamond problem, or ambiguity error because of allowing multiple inheritances with the default implementation in the interface method. Actually, it is a big problem for languages like C++ that allow multiple inheritances using classes. In C#, multiple inheritances using classes are not allowed but are allowed with interfaces. For a better understanding, please have a look at the below image.



Let us understand the diamond problem in C# using the interface. Please have a look at the below example. Here, interface A declares one method called **Method** and this method is by default abstract. Then this interface is inherited by interfaces B and C and both interfaces provide a default implementation for the method called **Method**. And finally, class D implements both B and C interfaces. Now, the diamond problem arises. **The interface member 'A.Method()' does not have the most specific implementation. Neither 'B.A.Method()', nor 'C.A.Method()'**. So, the compiler will get confused between the implementation from B and C interfaces and hence give you a compile-time error.

```

using System;
namespace DefaultInterfaceMethodsDemo
{
    interface A
    {
        void Method();
    }
    interface B : A
    {
        void A.Method()
        {
            System.Console.WriteLine("I am From Interface B");
        }
    }
    interface C : A
    {
        void A.Method()
        {
            System.Console.WriteLine("I am From Interface C");
        }
    }
    class D : B, C
    {
    }
    class Program
    {
        static void Main(string[] args)
        {
        }
    }
}

```

Here, we will get a compile-time error saying **Interface member 'A.Method()' does not have the most specific implementation. Neither 'B.A.Method()', nor 'C.A.Method()' are most specific** as shown in the below image.

4 references

interface A...

1 reference

interface B...

1 reference

interface C...

0 references

class D : B, C

```
{  
}
```

0 references

class Program

 interface Csharp8Features.B

CS8705: Interface member 'A.Method()' does not have a most specific implementation. Neither 'B.A.Method()', nor 'C.A.Method()' are most specific.

How to Solve the Diamond Problem with Interfaces in C#?

The .NET development team has decided to solve the Diamond Problem by taking the most specific override at runtime.

Diamonds with Classes: A class implementation of an interface member should always win over a default implementation in an interface, even if it is inherited from a base class. Default implementations are always a fallback only when the class does not have any implementation of that member at all.

Coming back to our example, the problem is that the most specific override cannot be inferred from the compiler. We solve this problem, by providing an implementation for the method “Method” in class D as shown in the below code, and now the compiler uses the class implementation to solve the diamond problem.

```
using System;  
namespace DefaultInterfaceMethodsDemo  
{  
    interface A  
    {  
        void Method();  
    }  
    interface B : A  
    {  
        void A.Method()  
        {  
            System.Console.WriteLine("I am From Interface B");  
        }  
    }  
    interface C : A
```

```

{
    void A.Method()
    {
        System.Console.WriteLine("I am From Interface C");
    }
}
class D : B, C
{
    // Now the compiler uses the most specific override, which is defined
class D.
    void A.Method()
    {
        System.Console.WriteLine("I am from class D");
    }
}
class Program
{
    static void Main(string[] args)
    {
        C c = new D();
        c.Method();
        Console.ReadKey();
    }
}
}

```

Output: I am from class D

Real-Time Example of Interface with Default Method Implementation in C#: Logging

The logger interface is a good example to explain the Default Method implementation of Inheritance in C#. In the below example, within the ILogger interface, we have declared one abstract method with the name WriteMessage and the rest of the all-other methods of the ILogger interface are provided with default implementations. The ConsoleLogger and TraceLogger classes are implementing the ILogger interface and provide implementation to the WriteMessage abstract method.

If you look at the below code, you can see that the code is compact and will not get any kind of warnings or errors. Before C# 8, it was mandatory to implement all the methods of an interface in a class unless that class is declared as an abstract class, and this might make your code DRY. The following example code is self-explained, so please go through the comment lines for a better understanding.

```

using System;
namespace DefaultInterfaceMethodsDemo
{
    //Creating an Enum to Store Log Message Type
    enum LogLevel
    {
        Information,
        Warning,
        Error
    }
}

```

```

}
//Creating the ILogger Interface
interface ILogger
{
    //The following Method is Abstract by Default
    void WriteMessage(LogLevel level, string message);
    //Interface Method with Default Implementation
    void WriteInformation(string message)
    {
        //Calling the WriteMessage to Log the actual Information message
        //WriteMessage Method is going to be executed from the Implementor
        WriteMessage(LogLevel.Information, message);
    }
    //Interface Method with Default Implementation
    void WriteWarning(string message)
    {
        //Calling the WriteMessage to Log the actual Warning message
        //WriteMessage Method is going to be executed from the Implementor
        WriteMessage(LogLevel.Warning, message);
    }
    //Interface Method with Default Implementation
    void WriteError(string message)
    {
        //Calling the WriteMessage to Log the actual Error message
        //WriteMessage Method is going to be executed from the Implementor
        WriteMessage(LogLevel.Error, message);
    }
}
//Implementor Class 1
//ConsoleLogger Class Inheriting from ILogger Interface
class ConsoleLogger : ILogger
{
    //Provide Implementation for the WriteMessage Abstract Method of ILogger
    Interface
    public void WriteMessage(LogLevel level, string message)
    {
        Console.WriteLine($"{level}: {message}");
    }
}
//Implementor Class 2
//TraceLogger Class Inheriting from ILogger Interface
class TraceLogger : ILogger
{
    //Provide Implementation for the WriteMessage Abstract Method of ILogger
    Interface
    public void WriteMessage(LogLevel level, string message)
    {
        Console.WriteLine($"{level}: {message}");
    }
}
class Program

```

```

{
    static void Main(string[] args)
    {
        //Create an Instance of ConsoleLogger Class and
        //Store the instance in ILogger Parent Reference Variable
        ILogger consoleLogger = new ConsoleLogger();
        //Call to the WriteWarning Method will be Executed from the ILogger
        //Internally WriteWarning Method will call the WriteMessage Method
        ConsoleLogger Class
        consoleLogger.WriteWarning("Console Logger Warning Message");
        //Call to the WriteInformation Method will be Executed from the ILogger
        Interface
        //Internally WriteInformation Method will call the WriteMessage Method
        ConsoleLogger Class
        consoleLogger.WriteInformation("Console Logger Information Message");
        //Create an Instance of TraceLogger Class and
        //Store the instance in ILogger Parent Reference Variable
        ILogger traceLogger = new TraceLogger();
        //Call to the WriteWarning Method will be Executed from the ILogger
        //Internally WriteWarning Method will call the WriteMessage Method
        TraceLogger Class
        traceLogger.WriteWarning("Trace Logger Warning Message");
        //Call to the WriteInformation Method will be Executed from the ILogger
        Interface
        //Internally WriteInformation Method will call the WriteMessage Method
        TraceLogger Class
        traceLogger.WriteInformation("Trace Logger Information Message");
        Console.ReadKey();
    }
}

```

Output:

```

Warning: Console Logger Warning Message
Information: Console Logger Information Message
Warning: Trace Logger Warning Message
Information: Trace Logger Information Message

```

Note: We can now add members to interfaces and provide an implementation for those members. This language feature enables API authors to add methods to an interface in later versions without breaking source or binary compatibility with existing implementations of that interface. Existing implementations inherit the default implementation.

In the next article, I am going to discuss [Pattern Matching Enhancements in C# 8](#) with Examples. Here, in this article, I try to explain **Default Interface Methods in C#** with Examples. I hope you enjoy this Default Interface Methods in C# with Examples article.